

Лабораторная работа №6

Поиск файлов. Перенаправление ввода-вывода. Просмотр
запущенных процессов

Mikhail Vzvarov

2026

Лабораторная работа №6

Поиск файлов. Перенаправление ввода-вывода.
Просмотр запущенных процессов

Цель работы

Ознакомление с инструментами поиска файлов и фильтрации текстовых данных. Приобретение практических навыков по управлению процессами, проверке использования диска и обслуживанию файловых систем.

Задание

1. Записать в файл file.txt названия файлов из /etc и домашнего каталога.
 2. Найти и записать в conf.txt все файлы с расширением .conf.
 3. Найти файлы в домашнем каталоге, начинающиеся с 'с'.
 4. Вывести файлы из /etc, начинающиеся с 'h'.
 5. Запустить процесс в фоновом режиме.
 6. Запустить и завершить gedit.
 7. Изучить команды df и du.
-

Выполнение работы

1. Запись файлов в file.txt `'''bash ls /etc > file.txt ls ~ » file.txt`
-

2. Запись файлов в file.txt В начале работы были созданы два файла с перечнем названий файлов из системного и домашнего каталогов.

```
'''bash ls /etc > file.txt
```

3. Поиск .conf файлов Для фильтрации записей в файле 'file.txt' и поиска файлов с расширением '.conf' использовалась команда 'grep':

```
'''bash grep ".conf" file.txt
```

Символ . в шаблоне позволяет экранировать точку, чтобы она интерпретировалась как литерал, а не как любой символ.

Для сохранения отфильтрованных записей в новый файл conf.txt использовалось перенаправление вывода: bash

```
grep ".conf" file.txt > conf.txt
```

4. Поиск файлов на 'с' Для поиска файлов в домашнем каталоге с именами, начинающимися на 'с', применялась команда 'find':

```
'''bash find ~ -maxdepth 1 -name "с*" -type f 2>/dev/null
```

Параметры:

-maxdepth 1 ограничивает поиск только текущим каталогом, без вложенных папок;

-name "с*" задаёт шаблон имени;

-type f ищет только обычные файлы;

2>/dev/null подавляет вывод сообщений об ошибках.

5. Поиск файлов на 'h' в /etc Аналогичным образом выполнялся поиск файлов в каталоге '/etc', имена которых начинаются на 'h':

```
'''bash find /etc -maxdepth 1 -name "h*" -type f 2>/dev/null
```

6. Фоновый процесс Для демонстрации работы с фоновыми процессами была запущена команда поиска файлов в фоновом режиме:

```
'''bash find /var/log -name "log*" -type f 2>/dev/null > ~/logfile &
```

Символ & в конце команды переводит её выполнение в фоновый режим, позволяя продолжить работу в консоли.

Параметры:

-name "log*" — имена файлов, начинающиеся с log;

-type f — только обычные файлы;

2>/dev/null — скрыть сообщения об ошибках;

> ~/logfile - перенаправить вывод в файл logfile в домашнем каталоге.

Для просмотра списка фоновых задач использовалась команда: bash

jobs

8. Использование df и du Для анализа использования дискового пространства были выполнены команды 'df' и 'du'.

df — просмотр свободного места на разделах диска:

'''bash df -h Флаг -h выводит размеры в “человекочитаемом” формате (ГБ, МБ).

du — оценка занимаемого места: bash

du -sh ~/

Параметры:

-s (summary) — показывает общий размер каталога, без перечисления вложенных файлов;

-h (human-readable) — выводит размер в удобном для чтения формате.

9. Поиск всех директорий в домашнем каталоге Для поиска всех директорий в домашнем каталоге использовалась команда 'find' с опцией '-type d':

'''bash find ~ -type d 2>/dev/null Параметры:

~ — начальный каталог поиска (домашний каталог);

-type d — искать только каталоги (директории);

2>/dev/null — подавляет вывод сообщений об ошибках (например, “Отказано в доступе”).

Для ограничения поиска только первым уровнем вложенности (без подкаталогов) используется опция -maxdepth 1: bash

find ~ -maxdepth 1 -type d 2>/dev/null

10. Завершение работы Все этапы лабораторной работы были выполнены в полном объёме. В ходе работы были получены практические навыки:

- Перенаправления потоков ввода-вывода ('>', '»', '2>');
- Фильтрации текстовых данных с помощью 'grep';
- Поиска файлов и каталогов с использованием 'find';
- Управления процессами (запуск в фоновом режиме, просмотр, завершение);
- Анализа использования дискового пространства с 'df' и 'du'.

Все созданные в процессе работы файлы ('file.txt', 'conf.txt', '~ /logfile') были проверены и при необходимости удалены.

```
'''bash rm ~/logfile rm file.txt conf.txt
```

Домашнее задание (Ответы на контрольные вопросы)

1. Какие потоки ввода-вывода вы знаете? Ответ: В системе существует три стандартных потока:

Поток	Описание	Файловый дескриптор
'stdin' Стандартный ввод (по умолчанию - клавиатура) 0 'stdout' 'stderr'	Стандартный вывод (по умолчанию — консоль)	1
	Стандартный вывод ошибок (по умолчанию — консоль)	2

2. Объясните разницу между операцией > и >>. Ответ: - '>' - перенаправляет вывод в файл, **перезаписывая** его содержимое (если файл существует, он будет заменён). - '>>' — перенаправляет вывод в файл, добавляя новые данные в конец файла (существующее содержимое сохраняется).

Примеры: '''bash echo "Новый текст" > file.txt # файл перезаписан echo "Дополнительный текст" » file.txt # добавлено в конец

3. Что такое конвейер? Ответ: Конвейер ('|') — это механизм в командной оболочке, позволяющий передавать вывод (stdout) одной команды на ввод (stdin) другой команды. Это позволяет строить цепочки команд для обработки данных без создания промежуточных файлов.

Пример: `'''bash ls -la | grep ".conf" | sort` В этом примере:

`ls -la` выводит список файлов;

`grep ".conf"` фильтрует строки, содержащие `.conf`;

`sort` сортирует полученный список.

Конвейеры можно объединять в длинные цепочки для комплексной обработки данных: `bash`

`ps aux | grep "python" | awk '{print $2}' | xargs kill -9`

Эта команда находит все процессы Python, извлекает их PID и завершает их.

5. Что такое PID и GID? Ответ:

- **PID** (Process IDentifier) — это уникальный числовой идентификатор, который присваивается каждому процессу в момент его создания. PID используется системой и пользователем для управления процессами (например, для завершения процесса через команду `kill`).
- **GID** (Group IDentifier) — это идентификатор группы процессов. В контексте управления заданиями GID позволяет обращаться к группе процессов, связанных общей задачей. При запуске процесса он получает тот же GID, что и родительский процесс.

Пример использования:

`'''bash ps aux | grep gedit` Вывод может содержать: `text`

`user 1234 0.0 0.5 ... gedit`

Здесь 1234 — это PID процесса `gedit`.

Для завершения процесса используется команда: `bash`

`kill 1234`

При работе с фоновыми задачами (jobs) используется идентификатор задачи (job ID), который обозначается как `%номер`: `bash`

`kill %1` Отличие PID от job ID: Идентификатор Описание PID Глобальный идентификатор процесса во всей системе Job ID Локальный идентификатор задачи в текущем сеансе терминала (обозначается `%номер`) Пример просмотра задач: `bash`

jobs

Вывод: text

[1] + running gedit &

Здесь [1] — это job ID, а сам процесс имеет свой PID, который можно узнать через ps.

7. Найдите информацию об утилитах top и htop. Каковы их функции? Ответ:

‘top’ — это встроенная интерактивная утилита для мониторинга системных процессов и ресурсов в реальном времени. Она показывает:

- Загрузку CPU (по ядрам и в целом);
- Использование оперативной памяти (RAM) и swap;
- Список запущенных процессов с их PID, состоянием, приоритетом, потреблением ресурсов;
- Общую информацию о системе (время работы, количество пользователей, нагрузку).

‘htop’ — это улучшенная версия ‘top’ с более удобным интерфейсом. Основные отличия:

Функция	‘top’	‘htop’
Интерфейс	Текстовый, монохромный	Цветной, с прокруткой
Управление	Только клавиатура (k, r, q и др.)	Клавиатура + поддержка мыши
Прокрутка	Нет	Да (можно прокручивать список процессов)
Завершение процессов	Требуется ввод PID	Можно выбрать процесс и нажать F9
Дерево процессов	Нет	Да (F5)
Настройка	Через конфигурационный файл	Интерактивная настройка (F2)

Пример запуска:

```
'''bash top
```

9. Можно ли по контексту (содержанию) найти файл? Если да, то как? Ответ: Да, можно. Для поиска файлов по их содержимому используется команда ‘grep’ с опцией рекурсивного поиска ‘-r’ (или ‘-R’). Это позволяет искать указанную строку или шаблон во всех файлах внутри каталога и его подкаталогов.

Основные опции ‘grep’ для поиска по содержимому:

Опция	Описание
‘-r’ или ‘-R’	Рекурсивный поиск по всем файлам в каталогах
‘-i’	Игнорировать регистр (поиск без учёта заглавных/строчных букв)
‘-l’ Вывести только имена файлов, содержащих совпадение ‘-n’	Показать номера строк, где найдено совпадение
‘-w’ Искать целые слова ‘--include=*.txt’	Искать только в файлах с указанным расширением
‘--exclude=*.log’	Исключить файлы с указанным расширением

Примеры использования:

1. **Поиск слова “error” во всех файлах в домашнем каталоге:**
`bash grep -r “error” ~/` Поиск с игнорированием регистра:

bash

grep -ri “error” ~/

Вывод только имён файлов, содержащих слово “pattern”:

bash

grep -rl “pattern” ~/

Поиск только в файлах с расширением .log:

bash

grep -r --include=*.log “error” /var/log/

Поиск с выводом номеров строк:

bash

grep -rn “error” ~/

Поиск целого слова (не части слова):

bash

```
grep -rw "error" ~/
```

Поиск в текущем каталоге, исключая файлы .log:

```
bash
```

```
grep -r --exclude=*.log "error" ./
```

Комбинация с другими командами: bash

```
find ~ -name "*.txt" -exec grep "pattern" {} ;
```

10. Как определить объём свободной памяти на жёстком диске?

Ответ: Для определения объёма свободного места на жёстком диске используется команда 'df' (disk free). Она показывает информацию о всех смонтированных файловых системах: размер, использованное и свободное пространство, процент заполнения и точку монтирования.

Синтаксис: `bash df [опции]`

11. Как определить объём вашего домашнего каталога? Ответ:

Для определения размера домашнего каталога используется команда 'du' (disk usage). Она показывает, сколько дискового пространства занимают файлы и каталоги.

Синтаксис: `bash du [опции] [путь]`

Выводы

В ходе выполнения лабораторной работы №6 я ознакомился с инструментами поиска файлов и фильтрации текстовых данных, а также приобрёл практические навыки по управлению процессами и заданиями, проверке использования диска и обслуживанию файловых систем.

В результате работы я научился:

1. Перенаправлять потоки ввода-вывода:

- Использовать '>' для перезаписи файлов;
- Использовать '»' для добавления данных в конец файла;
- Перенаправлять ошибки '2>' и комбинировать потоки.

2. Использовать конвейеры ('|') для построения цепочек команд, передавая вывод одной команды на ввод другой.

3. Применять команду 'find' для поиска файлов по различным критериям:

- По имени ('-name', '-iname');
 - По типу ('-type f/d');
 - По размеру ('-size');
 - По времени изменения ('-mtime').
4. **Использовать команду 'grep'** для фильтрации текстовых данных и поиска файлов по содержимому.
 5. **Управлять процессами:**
 - Запускать процессы в фоновом режиме ('&');
 - Просматривать список фоновых задач ('jobs');
 - Определять PID процессов (a'ps', 'pgrep');
 - Завершать процессы ('kill', 'pkill', 'killall').
 6. **Анализировать использование дискового пространства:**
 - Определять свободное место на разделах с помощью 'df -h';
 - Определять размер домашнего каталога с помощью 'du -sh ~/'.
 7. **Находить все директории** в домашнем каталоге с помощью 'find ~ -type d'.

Цель работы полностью достигнута. Полученные навыки являются фундаментальными для эффективной работы в командной строке Linux и будут полезны при администрировании систем и повседневной работе с файлами и процессами.

Библиография

1. GNU Bash Manual. — <https://www.gnu.org/software/bash/>
2. Linux man pages: find, grep, ps, kill, df, du, top, htop
3. Команды Linux. — <https://www.linux.org/>
4. The Linux Command Line. — William Shotts